

NGHIÊN CỨU MỘT SỐ THUẬT TOÁN TỐI ƯU BẬC HAI TRONG HUẤN LUYỆN MÔ HÌNH HỌC SÂU

RESEARCH ON SOME SECOND-ORDER OPTIMIZATION ALGORITHMS IN DEEP LEARNING MODEL TRAINING

ThS. Vũ Văn Đức¹, Nguyễn Khắc Hiếu², Trần Trọng Tùng², Trịnh Trung Hoàng²

¹ Khoa Công nghệ thông tin, Trường Đại học Kinh tế - Kỹ thuật Công nghiệp

² Lớp DHTI17A4HN, Trường Đại học Kinh tế - Kỹ Thuật Công nghiệp

Tóm tắt: Nghiên cứu này tập trung phân tích và so sánh các phương pháp tối ưu trong học sâu, bao gồm các thuật toán tối ưu bậc nhất và các thuật toán tối ưu bậc hai. Mục tiêu của nghiên cứu là làm rõ hiệu quả thực tiễn của các phương pháp tối ưu bậc hai trong huấn luyện mô hình học sâu, thông qua việc phân tích vai trò của thông tin độ cong và các kỹ thuật xấp xỉ ma trận Hessian. Phương pháp nghiên cứu bao gồm tổng hợp lý thuyết, xây dựng mô hình thử nghiệm và thực nghiệm trên máy tính. Kết quả cho thấy các thuật toán tối ưu bậc hai có tiềm năng cải thiện tốc độ hội tụ và độ ổn định trong các bài toán có điều kiện kém, dù đi kèm chi phí tính toán cao hơn. Nghiên cứu cung cấp cơ sở tham chiếu thực tiễn để lựa chọn thuật toán phù hợp với đặc điểm mạng nơ-ron và tài nguyên phần cứng.

Từ khóa: Tối ưu trong học sâu, Ma trận Hessian, Xấp xỉ Hessian, Tốc độ hội tụ, Thuật toán bậc hai.

Abstract: This study provides a comparative analysis of optimization methods in deep learning, specifically focusing on first-order and second-order optimization algorithms. The primary objective is to elucidate the practical efficacy of second-order methods in training deep learning models by analyzing the role of curvature information and Hessian matrix approximation techniques. The methodology incorporates a theoretical synthesis alongside empirical evaluations through computational experiments. The findings demonstrate that second-order algorithms can significantly enhance convergence speed and stability, particularly in ill-conditioned problems, albeit at the expense of higher computational costs. Ultimately, this research offers a practical reference framework for selecting optimal algorithms tailored to specific neural network architectures and available hardware resources.

Keywords: Deep Learning Optimization, Hessian Matrix, Hessian Approximation, Convergence Rate, Second-order Methods.

1. GIỚI THIỆU

Những năm gần đây chứng kiến sự bùng nổ mạnh mẽ của Học sâu, tạo tiền đề cho hàng loạt bước tiến đột phá trong các lĩnh vực như

thị giác máy tính, xử lý ngôn ngữ tự nhiên (NLP) và nhận dạng tiếng nói. Trong bối cảnh quy mô mô hình ngày càng mở rộng, hiệu quả huấn luyện phụ thuộc mật thiết vào việc lựa

chọn thuật toán tối ưu. Các phương pháp tối ưu bậc nhất hiện nay được áp dụng phổ biến nhờ chi phí tính toán thấp. Tuy nhiên, do chỉ dựa vào thông tin gradient (đạo hàm bậc một), nhóm phương pháp này có xu hướng gặp khó khăn trong việc khai thác cấu trúc hình học không gian tham số, đặc biệt tại các điểm yên ngựa, vùng bằng phẳng, hoặc gặp hiện tượng suy hao đạo hàm. Ngược lại, các phương pháp tối ưu bậc hai khai thác thông tin về độ cong của hàm mất mát thông qua ma trận Hessian (ma trận chứa các đạo hàm bậc hai), mang lại khả năng hội tụ nhanh và ổn định hơn. Tuy nhiên, thách thức về độ phức tạp tính toán và yêu cầu bộ nhớ lớn để xử lý ma trận Hessian đã hạn chế khả năng áp dụng của chúng trên các mạng nơ-ron sâu. Mặc dù đã có các nghiên cứu tổng quan về thuật toán tối ưu, phần lớn các tài liệu hiện nay chủ yếu tập trung vào việc đối chiếu lý thuyết hoặc thực nghiệm trên các hàm toán học tiêu chuẩn. Khoảng trống nghiên cứu cốt lõi hiện nay là sự thiếu hụt các đánh giá thực nghiệm toàn diện nhằm định lượng ranh giới đánh đổi giữa tài nguyên tính toán và hiệu suất hội tụ của các kỹ thuật xấp xỉ Hessian trên các kiến trúc mạng nơ-ron đặc thù trong thực tế. Để làm rõ vấn đề trên và là nội dung hỗ trợ cho các đề tài nghiên cứu sau này, nghiên cứu của chúng tôi không chỉ dừng lại ở việc tổng hợp lý thuyết mà còn tập trung phân tích, đánh giá thực nghiệm tính hiệu quả của các thuật toán tối ưu hai tiêu biểu so với các thuật toán tối ưu bậc nhất truyền thống. Giá trị gia tăng và đóng góp chính của bài báo bao gồm:

Thiết lập khung đánh giá thực nghiệm có đối chứng, nghiên cứu đã xây dựng một hệ thống thử nghiệm minh bạch, so sánh trực tiếp các thuật toán bậc nhất (SGD, Adam) với các thuật toán xấp xỉ bậc hai (L-BFGS, K-FAC) trên hai kiến trúc mạng nền tảng có đặc thù khác biệt là CNN (mạng tích chập) và RNN (mạng hồi quy) sử dụng bộ dữ liệu CIFAR-10.

Định lượng sự đánh đổi giữa hiệu suất và chi phí, bài báo cung cấp minh chứng bằng số liệu cụ thể về sự nghịch biến giữa tốc độ hội tụ và thời gian thực thi. Kết quả chỉ ra rằng K-FAC là một giải pháp cân bằng hiệu quả, vượt trội về độ chính xác trên cả CNN (63.45%) và RNN (50.90%) so với SGD và Adam, mặc dù đòi hỏi chi phí thời gian huấn luyện cao hơn.

Xác định ranh giới tính ổn định theo kiến trúc mô hình, nghiên cứu phát hiện ra tính phụ thuộc mạnh mẽ của thuật toán xấp xỉ bậc hai vào cấu trúc mạng. Điển hình, L-BFGS thể hiện sự biến thiên lớn khi đạt hiệu suất cao nhất trên mô hình CNN (64.63%) nhưng lại suy giảm tính ổn định và hiệu năng nghiêm trọng trên mô hình RNN (chỉ đạt 33.19%). Phát hiện này cung cấp cơ sở tham chiếu quan trọng cho việc lựa chọn thuật toán phù hợp với từng bài toán cụ thể

2. TỔNG QUAN BÀI TOÁN TRONG TỐI ƯU HỌC SÂU

Mục tiêu cốt lõi của Học sâu là tối thiểu hóa một hàm mất mát (loss function) trong không gian trọng số với nhiều chiều. Hàm mất mát là một hàm toán học dùng để *định lượng mức độ sai lệch giữa đầu ra dự đoán của mô hình và giá trị thực* trong quá trình huấn luyện. Giá trị của hàm mất mát càng nhỏ thì mô hình càng dự đoán chính xác. Trong học sâu, hàm mất mát đóng vai trò trung tâm trong việc *định hướng quá trình tối ưu*, bởi các thuật toán huấn luyện sẽ điều chỉnh tham số của mạng nơ-ron sao cho giá trị hàm này được cực tiểu hóa. Việc lựa chọn hàm mất mát phù hợp có ảnh hưởng trực tiếp đến tốc độ hội tụ, độ ổn định và chất lượng cuối cùng của mô hình.

Bài toán được mô hình hóa thành việc tìm kiếm tập hợp tham số θ tối ưu để giảm thiểu rủi ro thực nghiệm: [1]

$$\min \tilde{L}(\theta) = \frac{1}{N} \sum_{i=1}^N L(y(x, \theta), t) + R(\theta) \quad (1)$$

Mục tiêu: Tìm một điểm cực trị hoặc giải pháp tối ưu bằng các kỹ thuật tối ưu hóa dựa trên

gradient, sử dụng thông tin bậc nhất hoặc bậc hai để cập nhật tham số trọng số.

Bề mặt lỗi (Loss Surface) trong học sâu thường rất phức tạp, tạo ra nhiều thách thức cho quá trình tối ưu hóa, đặc biệt đối với các hàm mất mát trên các không gian tham số liên tục, nhiều chiều. Tính chất phi lồi của bề mặt lỗi khiến cho bài toán trở nên khó khăn. Các kỹ thuật tối ưu hóa phi lồi nhiều chiều thường không đảm bảo về mặt lý thuyết nhưng vẫn đạt được kết quả tối ưu trên thực tế.

Điểm yên ngựa: Bề mặt lỗi thường rất phức tạp với sự tồn tại các điểm yên ngựa và các vùng phẳng. Điểm dừng có giá trị hàm mất mát lớn rất có khả năng là điểm yên ngựa. Sự tồn tại của một số lượng lớn các điểm yên ngựa có thể làm chậm đáng kể quá trình học tập vì chúng là những vùng có đạo hàm (Gradient) xấp xỉ bằng 0 ở mọi hướng với giá trị lỗi cao (Plateaus), tạo cảm giác như có một điểm cực tiểu cục bộ và ngoài ra, là do các điểm tối hạn này được bao quanh bởi các Plateaus có độ cong nhỏ.

Nhiều của Gradient: Các phương pháp tối ưu hóa phổ biến sử dụng ước tính gradient dựa trên các nhóm mẫu nhỏ (mini-batch), nhiều gradient đóng vai trò như một cơ chế điều chuẩn ngầm, giúp mô hình tránh các cực tiểu nhọn và hội tụ vào các cực tiểu phẳng, vốn được cho là có khả năng tổng quát hóa tốt hơn, tức là đạt hiệu năng cao không chỉ trên tập huấn luyện mà còn trên dữ liệu chưa từng thấy.

Do cấu trúc phức tạp của hàm mất mát gây ảnh hưởng trực tiếp đến hiệu quả của các thuật toán tối ưu, sự mất cân đối về tốc độ thay đổi của Gradient theo các hướng khác nhau. Các thuật toán tối ưu bậc nhất rất nhạy cảm với các vấn đề này và thường dễ bị mắc kẹt tại các "cực tiểu giả" do không thể điều hướng khi bề mặt có độ cong cao và thấp cùng lúc. Các phương pháp tối ưu bậc hai hoặc bậc nhất cải tiến được đề xuất để khắc phục các vấn đề trên bằng cách mô hình hóa cấu trúc hình học cục

bộ của hàm mất mát và điều chỉnh hướng tìm kiếm cho phù hợp, hiệu quả hơn.

Mặc dù các phương pháp tối ưu bậc nhất được sử dụng rộng rãi trong huấn luyện mạng nơ-ron sâu nhờ chi phí tính toán thấp và khả năng mở rộng tốt, chúng vẫn tồn tại nhiều hạn chế, đặc biệt trong các bài toán có không gian tham số lớn và hàm mất mát. Những hạn chế này là động lực thúc đẩy việc nghiên cứu và phát triển các phương pháp tối ưu bậc hai và xấp xỉ bậc hai, nhằm khai thác thông tin độ cong của hàm mất mát để cải thiện tốc độ và độ ổn định của quá trình huấn luyện mạng nơ-ron sâu.

3. MỘT SỐ THUẬT TOÁN TỐI ƯU BẬC NHẤT VÀ TỐI ƯU BẬC HAI

3.1. Thuật toán tối ưu bậc nhất

3.1.1. Thuật toán SGD

SGD là nền tảng của tối ưu hóa mạng nơ-ron, thay vì tính toán trên toàn bộ dữ liệu. Việc sử dụng mini-batch tạo ra nhiễu giúp mô hình có khả năng thoát khỏi các điểm cực tiểu nhọn và tìm đến các điểm cực tiểu phẳng. nó cập nhật tham số θ dựa trên gradient g_t của một nhóm nhỏ dữ liệu tại bước lặp t .

Quy tắc cập nhật cơ bản tại bước lặp t : [1]

$$g_t = \nabla_{\theta} \left(\frac{1}{m} \sum_{i=1}^m L(f(x^{(i)}; \theta), y^{(i)}) \right) \quad (2)$$

$$\theta_{t+1} = \theta_t - \epsilon g_t$$

g_t : Gradient ước lượng từ mini-batch.

ϵ : Tốc độ học (learning rate).

Tuy nhiên, SGD thường hội tụ chậm và dễ bị dao động mạnh tại các vùng cực tiểu nhọn hoặc điểm yên ngựa do tỷ lệ tốc độ học cố định cho mọi tham số, việc chọn tốc độ học phù hợp rất khó khăn và thường yêu cầu kỹ thuật điều chỉnh tốc độ học. [1] [2]

3.1.2 RMSprop

Bằng cách chia gradient cho căn bậc hai của trung bình bình phương gradient, RMSprop tự động giảm tốc độ học ở các chiều có độ dốc lớn và tăng ở chiều có độ dốc nhỏ. Điều này

giúp thuật toán hội tụ nhanh và ổn định hơn SGD thuần túy. Hoạt động tốt trong các mạng nơ-ron sâu và RNN, nơi bề mặt lỗi phức tạp [1] [5]

Quy tắc cập nhật: Thuật toán duy trì trung bình di động theo hàm mũ của bình phương gradient: [1]

$$r_t = \rho r_{t-1} + (1 - \rho) g_t^2 \quad (3)$$

$$\Delta \theta = - \frac{\epsilon}{\sqrt{r_t + \delta}} \times g_t$$

r : Biến tích lũy bình phương gradient.

ρ : Tỷ lệ suy giảm, thường là 0.9.

δ : Hằng số nhỏ (ví dụ 10^{-8}). [1]

RMSprop không có cơ chế hiệu chỉnh sai lệch (bias correction) cho các bước đầu tiên khi biến tích lũy r được khởi tạo bằng 0, điều này có thể gây ra mức thay đổi tham số quá lớn ở các bước lặp ban đầu.

3.1.3 Thuật toán Adam

Adam kết hợp sử dụng đà quán tính (moment) và RMSprop, Adam khá bền vững với việc lựa chọn siêu tham số và thường hoạt động tốt với các giá trị mặc định, thuật toán cập nhật tham số không bị ảnh hưởng bởi việc nhân gradient với một hằng số. [1] Nhờ cơ chế hiệu chỉnh sai lệch, Adam đạt tiến độ nhanh trong giai đoạn đầu huấn luyện [6]

Công thức cập nhật: Adam duy trì ước lượng của moment bậc một (s) và moment bậc hai (r): [1], [6]

$$s_t = \rho_1 s_{t-1} + (1 - \rho_1) g_t \quad (4)$$

$$r_t = \rho_2 r_{t-1} + (1 - \rho_2) g_t^2$$

Thực hiện hiệu chỉnh sai lệch do khởi tạo bằng 0:

$$\hat{s}_t = \frac{s_t}{1 - \rho_1^t}; \hat{r}_t = \frac{r_t}{1 - \rho_2^t} \quad (5)$$

Tham số được cập nhật: $\Delta \theta = -\epsilon \frac{\hat{s}_t}{\sqrt{\hat{r}_t + \delta}}$. [1]

ρ_1 (hoặc β_1): Hệ số suy giảm cho moment bậc một (thường là 0.9) [2]

ρ_2 (hoặc β_2): Hệ số suy giảm cho moment bậc hai (thường là 0.999) [2]

ϵ (learning rate): Thường khởi tạo khoảng 10^{-3} hoặc 10^{-4} [1] [2]

Adam có xu hướng hội tụ đến các cực tiểu nhọn, làm suy giảm khả năng tổng quát hóa và dẫn đến sai số kiểm thử cao hơn so với SGD khi được tinh chỉnh cẩn thận. Ngoài ra, Adam yêu cầu bộ nhớ lớn hơn để lưu trữ các biến trung gian (moment bậc nhất và bậc hai), và hiệu năng của thuật toán vẫn nhạy cảm với việc lựa chọn siêu tham số, đặc biệt là tốc độ học ban đầu và các hệ số suy giảm. [1][6]

3.2 Phương pháp tối ưu bậc hai

Các thuật toán tối ưu hóa bậc hai được thiết kế để sử dụng cấu trúc hình học của hàm mất mát nhằm tăng tốc độ hội tụ so với các phương pháp bậc nhất. Về mặt lý thuyết, bằng cách sử dụng ma trận Hessian, các thuật toán này có thể xác định hướng cập nhật tham số tối ưu (bằng cách thay đổi tỷ lệ gradient với nghịch đảo của Hessian), qua đó cho phép hội tụ trực tiếp đến nghiệm tối ưu trong trường hợp hàm mục tiêu là hàm bậc hai. [1] [3]

3.2.1 Phương pháp Newton

Là nền tảng lý thuyết cho các thuật toán bậc hai, sử dụng thông tin độ cong chính xác để tìm hướng di chuyển tối ưu. Về lý thuyết, phương pháp Newton cổ điển sử dụng chính xác ma trận Hessian, không thực hiện xấp xỉ. [1]

Công thức & Ý tưởng chính: Phương pháp Newton dựa trên khai triển Taylor bậc hai của hàm mất mát $f(x)$. Quy tắc cập nhật tham số θ tại bước k là: $\theta_{k+1} = \theta_k - H^{-1} g_k$ [1]

Trong đó: H là ma trận Hessian ($H = \nabla^2 f$) và g là vector gradient (∇f). Ý tưởng cốt lõi là thay đổi tỷ lệ gradient theo từng hướng bằng nghịch đảo của Hessian, cho phép quá trình tính toán trực tiếp hội tụ đến điểm cực tiểu của hàm bậc hai xấp xỉ chỉ trong một bước. [1]

Xấp xỉ Hessian: Về lý thuyết, phương pháp Newton cổ điển sử dụng chính xác ma trận

Hessian, không thực hiện xấp xỉ. Tuy nhiên, trong thực tế với học sâu, việc tính toán chính xác này thường bất khả thi do kích thước quá lớn. [1]

Ưu điểm:

Hội tụ nhanh: Tốc độ giảm sai số của thuật toán khi tiến gần đến điểm cực tiểu (Ở mỗi bước lặp, sai số của bước sau tỷ lệ thuận với bình phương sai số của bước trước) [1]

Bất biến với tỷ lệ: Không bị ảnh hưởng bởi phép biến đổi tuyến tính của tham số, giúp giải quyết vấn đề độ cong mà các phương pháp bậc một gặp phải. [3]

Chi phí & Nhược điểm:

Chi phí tính toán lớn: Việc tính toán và nghịch đảo ma trận Hessian tăng nhanh theo số lượng tham số với độ phức tạp $O(N^3)$ về thời gian và $O(N^2)$ về bộ nhớ, không khả thi đối với các mô hình quy mô lớn. [1]

Thu hút vào điểm yên ngựa: Nếu Hessian không xác định dương, dùng Newton thuần túy có thể khiến thuật toán bị thu hút mạnh về phía các điểm yên ngựa dẫn đến dừng tại những nghiệm không tối ưu (nơi giá trị hàm mất mát cao), thay vì tiếp tục tối ưu hướng tới các điểm cực tiểu cục bộ. [1]

3.2.2. Phương pháp BFGS & L-BFGS

Đây là các phương pháp Quasi-Newton phổ biến nhất, cố gắng giữ lại ưu điểm của Newton mà không cần tính Hessian trực tiếp.

Công thức & Ý tưởng chính: BFGS không tính trực tiếp H^{-1} mà xây dựng ma trận xấp xỉ nghịch đảo Hessian ($B_k \approx H^{-1}$) qua từng bước lặp, dựa trên sự thay đổi của tham số và gradient.

Cập nhật dựa trên phương trình secant: [1]

$$B_{k+1}s_k = y_k \quad (6)$$

$s_k = \theta_{k+1} - \theta_k$: thay đổi tham số.

$y_k = g_{k+1} - g_k$: thay đổi gradient.

Xấp xỉ Hessian:

BFGS: Duy trì một ma trận B_k đầy đủ kích thước $N \times N$. Cập nhật B_{k+1} từ B_k bằng một công thức hạng hai sử dụng các cặp vector (s_k, y_k) .

L-BFGS: Không lưu trữ ma trận B_k . Thay vào đó, nó chỉ lưu trữ m cặp vector (s, y) gần nhất (thường $m < 20$) để tính toán ngầm định tích $B_k g_k$.

Ưu điểm:

Nhờ cơ chế xấp xỉ thông tin bậc hai, các phương pháp này đạt được tốc độ hội tụ siêu tuyến tính, vượt trội so với các phương pháp gradient bậc nhất truyền thống. L-BFGS giúp giảm đáng kể yêu cầu bộ nhớ, hạ độ phức tạp lưu trữ xuống mức tuyến tính $O(N)$, qua đó phù hợp hơn với các mô hình có số lượng tham số lớn so với BFGS gốc. [1]

Nhược điểm / Chi phí:

BFGS gốc yêu cầu độ phức tạp $O(N^2)$ về bộ nhớ khiến việc áp dụng trực tiếp trong huấn luyện mạng nơ-ron sâu trở nên không khả thi. L-BFGS có thể gặp hiện tượng hội tụ chậm hoặc thiếu ổn định trong các bài toán học sâu, đặc biệt khi cấu trúc của ma trận Hessian thay đổi nhanh theo thời gian huấn luyện. [1]

3.2.3 Phương pháp Hessian-Free (HF)

Đây là phương pháp Newton cắt cụt, được thiết kế đặc biệt để huấn luyện mạng nơ-ron sâu và mạng RNN mà không cần tạo ma trận Hessian.

Công thức & Ý tưởng chính: HF giải bài toán tối ưu bậc hai địa phương $Bp = -g$ (tìm hướng cập nhật p) bằng thuật toán Linear Conjugate Gradient (CG) thay vì nghịch đảo ma trận trực tiếp. Quy trình này chạy một vòng lặp CG bên trong mỗi bước cập nhật tham số. [3]

Xấp xỉ Hessian: Hessian-vector product: HF không tính ma trận B , HF tính tích của ma trận với một vector (Bv) . Việc này được thực hiện chính xác và hiệu quả bằng thuật toán của Pearlmutter (một dạng tính đạo hàm theo

hướng) hoặc sai phân hữu hạn, với chi phí bằng một lần tính gradient. [3]

Gauss-Newton Matrix (G): Trong thực tế, HF thường sử dụng ma trận Gauss-Newton (G) thay vì Hessian (H) để đảm bảo ma trận luôn xác định dương, giúp thuật toán CG ổn định và tránh bị hút vào điểm yên ngựa.

Ưu điểm:

Xử lý độ cong xấu: Tối ưu rất tốt trong các *hẻm núi hẹp* (Mô tả các khu vực trên bề mặt hàm mất mát có độ cong mất cân đối giữa các hướng nơi hàm mất mát biến thiên mạnh theo một số chiều trong không gian tham số nhưng gần như phẳng theo các chiều còn lại) và giảm vấn đề mất gradient trong RNN. [3]

Tiết kiệm bộ nhớ: Không cần lưu ma trận Hessian, chỉ dùng vector. [3]

Nhược điểm / Chi phí:

Chi phí tính toán trên mỗi bước cập nhật tương đối cao, do mỗi lần cập nhật yêu cầu thực hiện nhiều vòng lặp Conjugate Gradient (CG), trong đó mỗi vòng lặp có chi phí tương đương với một lần *lan truyền ngược* (lan truyền ngược cho phép xác định mức độ ảnh hưởng của từng trọng số đến sai số đầu ra, từ đó cung cấp gradient cần thiết cho các thuật toán tối ưu để cập nhật tham số và giảm giá trị hàm mất mát). Bên cạnh đó, HF thường đòi hỏi kích thước batch lớn để ước lượng ổn định thông tin độ cong, điều này làm giảm tần suất cập nhật tham số so với các phương pháp bậc nhất như SGD, từ đó ảnh hưởng đến hiệu quả huấn luyện trong thực tế. [3]

3.2.4 Phương pháp K-FAC

K-FAC là một phương pháp xấp xỉ hiệu quả cho mạng nơ-ron, cố gắng cân bằng giữa chi phí tính toán của phương pháp bậc một và tốc độ hội tụ của bậc hai.

Công thức & Ý tưởng chính: K-FAC xấp xỉ ma trận Fisher (hoặc Hessian) dưới dạng một ma trận khối chéo, trong đó mỗi khối tương ứng với các tham số của một lớp trong mạng nơ-ron. [4]

Xấp xỉ Hessian: Mỗi khối Hessian F_l của lớp l được xấp xỉ bằng tích Kronecker của hai ma trận nhỏ hơn:

$$F_l \approx A_{l-1} \otimes G_l. (7)$$

Dựa trên tính chất đại số tuyến tính:

$$(A \otimes G)^{-1} = A^{-1} \otimes G^{-1} (8)$$

Nghịch đảo của khối này có thể tính rất nhanh mà không cần nghịch đảo toàn bộ ma trận lớn. Điều đó giảm độ phức tạp từ $O(N^3)$ của phương pháp Newton xuống mức chi phí thấp. [7]

Ưu điểm: K-FAC có yêu cầu bộ nhớ tương đương các phương pháp bậc nhất như Adam, do chỉ cần lưu trữ và xử lý các ma trận kích thước nhỏ tương ứng với các xấp xỉ Kronecker của ma trận Fisher. Nhờ khai thác hiệu quả thông tin bậc hai xấp xỉ, K-FAC thường đạt tốc độ hội tụ nhanh hơn so với SGD và Adam trong huấn luyện các mạng nơ-ron tích chập sâu, điển hình là các kiến trúc như ResNet. [4]

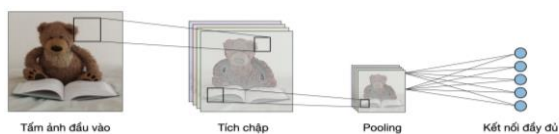
Nhược điểm / Chi phí:

So với SGD, K-FAC có độ phức tạp cài đặt cao hơn, do cần duy trì và cập nhật các ma trận xấp xỉ, cũng như thực hiện các phép nghịch đảo ma trận tương ứng. Mặc dù chi phí tính toán thấp hơn đáng kể so với việc xử lý Hessian đầy đủ, K-FAC vẫn đòi hỏi chi phí tính toán cao hơn một mức nhất định so với các phương pháp bậc nhất truyền thống.

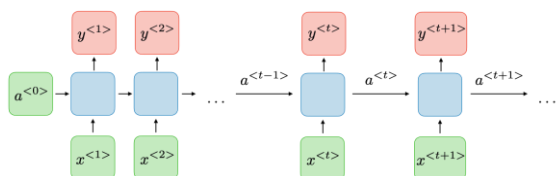
4. THỰC NGHIỆM

Nghiên cứu tiến hành thực nghiệm huấn luyện trên kiến trúc mạng nơ-ron tích chập (CNN) và mô hình mạng nơ-ron hồi quy (RNN) trên bộ dữ liệu chuẩn CIFAR-10 (Bộ dữ liệu CIFAR-10 gồm 60.000 ảnh màu kích thước 32×32 thuộc 10 lớp khác nhau. Dữ liệu được chia thành 50.000 ảnh cho tập huấn luyện và 10.000 ảnh cho tập kiểm thử). So sánh dựa trên độ giảm giá trị loss (Đại lượng số đo mức độ sai lệch giữa đầu ra dự đoán của mô hình và giá trị thực) sau mỗi epoch (Đơn vị thời gian huấn luyện, dùng để chỉ một

lần mô hình được học trên toàn bộ tập dữ liệu huấn luyện) và thời gian.



Hình 1. Kiến trúc mạng nơ-ron tích chập (CNN)



Hình 2. Kiến trúc mạng nơ-ron hồi quy RNN

Môi trường thực nghiệm và dữ liệu:

Thực nghiệm được triển khai trên nền tảng PyTorch với GPU NVIDIA RTX 2050 (4GB VRAM). Tham số *random seed* được cố định ở giá trị 42 nhằm đảm bảo tính tái lập tuyệt đối. Nghiên cứu đánh giá 4 thuật toán tối ưu (SGD, Adam, L-BFGS, K-FAC) trên hai kiến

trúc mạng đặc thù là CNN và RNN. Về kiến trúc, mạng CNN được thiết kế tinh gọn để tập trung đánh giá năng lực tối ưu thay vì sức mạnh trích xuất đặc trưng. Cấu hình gồm hai lớp tích chập (6 và 16 kernel 5×5 , hàm kích hoạt ReLU, Max Pooling (2×2) và một lớp fully connected phân loại 10 đầu ra, với tổng số 6.882 tham số huấn luyện. Mạng Vanilla RNN được thiết lập với kích thước đầu vào 96, 128 đơn vị ẩn và 10 nút đầu ra. Đối với thiết lập huấn luyện, tốc độ học được tinh chỉnh qua phương pháp thử nghiệm dạng lưới, riêng Adam giữ nguyên tham số mặc định để đánh giá hành vi hội tụ điển hình. Đáng chú ý, L-BFGS được vận hành ở chế độ *full-batch* thay vì *mini-batch*. Do đặc thù rất nhạy cảm với nhiễu gradient của phương pháp Quasi-Newton, chạy *full-batch* là điều kiện bắt buộc để phương trình xấp xỉ Hessian không bị sai lệch. Riêng với mô hình RNN, để tránh tràn bộ nhớ khi xử lý *full-batch*, thuật toán L-BFGS được huấn luyện trên một tập dữ liệu con ngẫu nhiên gồm 5.000 mẫu.

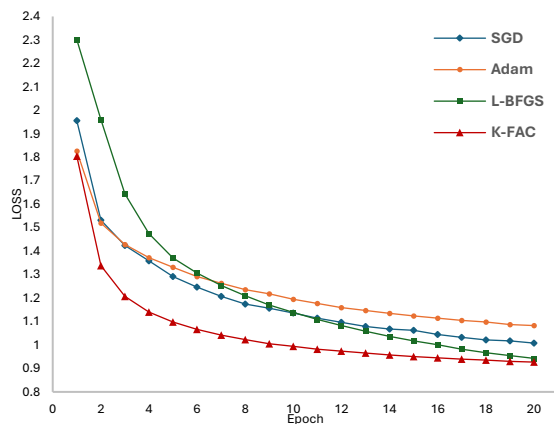
Bảng 1. Bảng cấu hình tham số 4 thuật toán với kiến trúc CNN

Thuật toán	Learning Rate	Tham số khác	Ghi chú kỹ thuật
SGD	CNN: 0.01 RNN: 0.05	Momentum = 0.9	Baseline bậc 1 cơ bản. Dùng đà quán tính để giảm dao động hội tụ.
Adam	CNN: 0.001 RNN: 0.001	$\beta_1 = 0.9, \beta_2 = 0.999$	Baseline bậc 1 hiện đại. Giữ nguyên tham số mặc định để đánh giá hành vi hội tụ điển hình.
K-FAC	CNN: 0.002 RNN: 0.01	CNN: Damping = 0.05, Update Freq = 20 RNN: Damping = 0.1, Update Freq = 50	Khai thác thông tin độ cong cục bộ. Tần suất cập nhật nghịch đảo ma trận được thiết lập riêng để cân bằng chi phí tính toán.
L-BFGS	CNN: 1.0 RNN: 0.1	CNN: Max Iter = 20 RNN: Max Iter = 5 (History Size: 10)	Chạy chế độ Full-batch để xấp xỉ Hessian. Lưu ý: Huấn luyện RNN trên tập dữ liệu con (5000 mẫu) để tối ưu bộ nhớ.

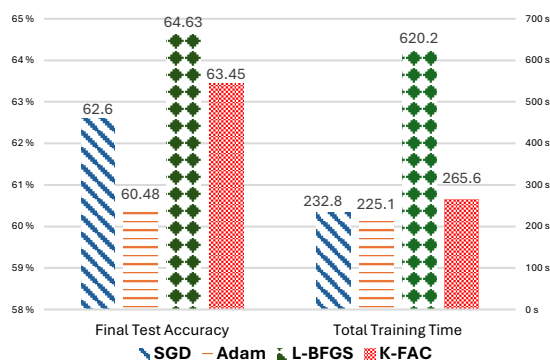
Để đánh giá hiệu năng, quy trình huấn luyện được thiết lập riêng biệt cho từng kiến trúc. Với mạng CNN, thực nghiệm diễn ra trong 20 epoch, kích thước lô (*batch size*) là 256. Thiết lập cỡ mẫu lớn này đặc biệt quan trọng để thuật toán K-FAC ước lượng ma trận Fisher/Hessian với phương sai thấp, giúp cân bằng giữa nhiễu gradient và độ ổn định. Trong khi đó, kiến trúc RNN được huấn luyện 15 epoch với kích thước lô 128. Đặc biệt, để kiểm soát hiện tượng bùng nổ gradient trong quá trình lan truyền ngược qua thời gian, kỹ thuật cắt gradient (*gradient clipping*) với chuẩn tối đa bằng 1.0 đã được áp dụng thống nhất cho tất cả các thuật toán.

4.1. Kết quả và đánh giá thực nghiệm

Kết quả mô hình với kiến trúc mạng CNN:

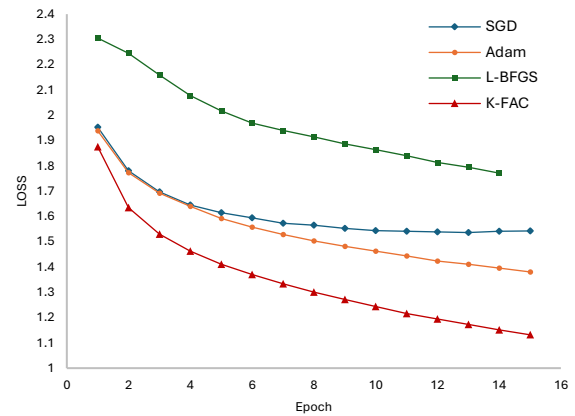


Hình 3. Đồ thị biểu diễn sự hội tụ của hàm mất mát với mô hình simpleCNN

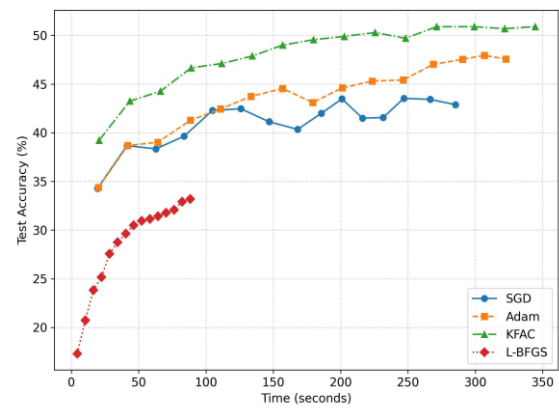


Hình 4. So sánh độ chính xác trên tập kiểm thử và tổng thời gian huấn luyện mô hình simpleCNN

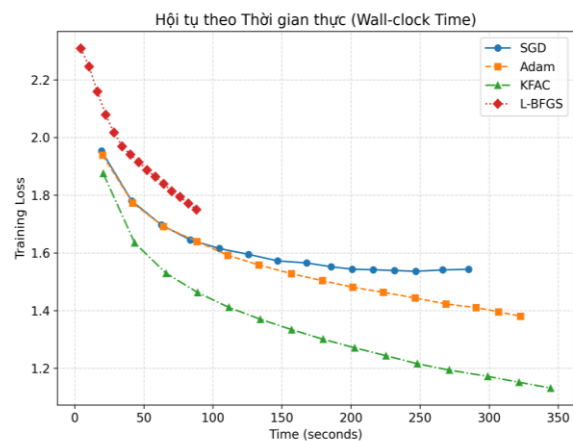
Kết quả mô hình với kiến trúc mạng RNN:



Hình 5. Đồ thị biểu diễn sự hội tụ của hàm mất mát với mô hình RNN



Hình 6. Biểu đồ độ chính xác theo thời gian thực của các thuật toán với mô hình RNN



Hình 7. Biểu đồ hội tụ theo thời gian thực của các thuật toán với mô hình RNN

Từ những kết quả trên cho ra bảng số liệu tổng quát được thể hiện ở bảng 2:

Bảng 2. Bảng tổng hợp kết quả giữa 4 thuật toán huấn luyện với kiến trúc mạng CNN và RNN

Thuật toán	Loss cuối		Độ chính xác (%)		Thời gian (s)	
	CNN	RNN	CNN	RNN	CNN	RNN
SGD	1.0079	1.5434	62.60	42.88	232.8	229.8
Adam	1.0826	1.3805	60.48	47.57	225.1	229.2
L-BFGS	0.9427	1.7503	64.63	33.19	620.2	87.2
K-FAC	0.9270	1.1379	63.45	50.90	265.6	338.0

Thứ nhất, kết quả thực nghiệm minh chứng rõ rệt sự nghịch biến giữa khả năng tối ưu trên từng bước lặp và chi phí thời gian thực thi. Xét theo số lượng epoch, các phương pháp bậc hai (K-FAC, L-BFGS) thể hiện tốc độ giảm hàm mất mát nhanh và sâu hơn hẳn so với SGD hay Adam. Tuy nhiên, khi đối chiếu theo thời gian thực, lợi thế này bị thu hẹp đáng kể. Cụ thể trên mạng CNN, K-FAC đạt độ chính xác 63.45% nhưng tiêu tốn 265.6 giây, cao hơn so với SGD (~232.8s) và Adam (~225.1s). Trên RNN, thời gian của K-FAC (338.0s) cũng cao hơn đáng kể so với nhóm bậc nhất (SGD: 229.8s, Adam: 229.2s). Sự chênh lệch này bắt nguồn trực tiếp từ chi phí tính toán các xấp xỉ Kronecker và nghịch đảo ma trận khối tại mỗi lần cập nhật của K-FAC.

Thứ hai, đối với mạng CNN, quá trình huấn luyện sử dụng kích thước lô lớn là 256. Thiết lập này đặc biệt quan trọng đối với K-FAC. Khác với các phương pháp bậc nhất có thể hoạt động tốt với nhiều, thuật toán xấp xỉ bậc hai rất nhạy cảm với độ lệch của gradient. Kích thước mẫu lớn giúp đảm bảo ước lượng ma trận Fisher/Hessian có phương sai thấp, giữ cho hướng cập nhật của K-FAC ổn định và không bị chệch hướng do nhiễu dữ liệu.

Thứ ba, trong quá trình thực nghiệm chúng tôi nhận thấy sự suy giảm tính ổn định của L-BFGS trên kiến trúc RNN, sự mất ổn định của phương pháp L-BFGS phụ thuộc mạnh vào

kiến trúc mô hình. Mặc dù L-BFGS đạt hiệu năng tốt nhất trên CNN (64.63%), thuật toán này lại suy giảm độ chính xác nghiêm trọng xuống chỉ còn 33.19% trên mô hình RNN. Nguyên nhân là do cấu trúc chuỗi thời gian của RNN tạo ra bề mặt lỗi thay đổi cực kỳ phi tuyến. Cơ chế của L-BFGS là xấp xỉ nghịch đảo Hessian dựa trên lịch sử của một số bước cập nhật (history size: 10). Trong không gian tham số của RNN, cấu trúc độ cong thay đổi quá nhanh chóng khiến thông tin xấp xỉ từ các bước trước lập tức trở nên lỗi thời, đẩy quá trình tối ưu đi sai hướng dù đã áp dụng kỹ thuật cắt gradient. Do rào cản về tài nguyên tính toán (thiết bị sử dụng GPU 4GB VRAM) và chi phí thời gian huấn luyện rất lớn của các thuật toán bậc hai, việc chạy lặp lại thực nghiệm nhiều lần để tính độ lệch chuẩn là không khả thi. Để giải quyết vấn đề này và đảm bảo tính công bằng, nghiên cứu đã chủ động cố định tham số random seed ở giá trị 42 cho mọi thuật toán. Điều này đảm bảo tính tái lập tuyệt đối của bề mặt hàm mất mát, điểm khởi tạo trọng số và thứ tự phân chia dữ liệu, cho phép quỹ đạo hội tụ phản ánh chính xác bản chất năng lực của từng phương pháp tối ưu.

5. KẾT LUẬN

Kết quả thực nghiệm trên bộ dữ liệu CIFAR-10 cho thấy các phương pháp tối ưu bậc hai mang lại khả năng hội tụ với số vòng lặp ít

hơn, nhưng đi kèm chi phí thời gian tính toán nghịch đảo ma trận cao so với các phương pháp bậc nhất. Đặc biệt, thuật toán K-FAC thể hiện hiệu quả cân bằng đáng chú ý, bộc lộ sự vượt trội về chất lượng hội tụ và duy trì được sự ổn định trên cả hai kiến trúc có đặc tính phi tuyến khác biệt (CNN và RNN). Tuy nhiên, nghiên cứu cũng xác nhận rằng hiệu năng của các thuật toán xấp xỉ Quasi-Newton (điển hình như L-BFGS) bị giới hạn và phụ thuộc nghiêm trọng vào cấu trúc bề mặt lỗi của từng loại mạng. Trong tương lai, khi rào cản về phần cứng được nới lỏng, các nghiên cứu tiếp theo cần tiến hành đánh giá trên các tập dữ

liệu và mô hình ngôn ngữ quy mô lớn hơn để kiểm chứng giới hạn chịu đựng của kỹ thuật xấp xỉ Kronecker.

Đặc biệt, một hướng phát triển tiềm năng là nghiên cứu các phương pháp lai – kết hợp tính linh hoạt, chi phí thấp của các phương pháp tối ưu bậc nhất thích nghi (adaptive first-order methods như Adam) với cơ chế hiệu chỉnh độ cong từ K-FAC. Hướng đi này hứa hẹn tận dụng được tốc độ xử lý mini-batch của đạo hàm bậc một đồng thời thừa hưởng độ chính xác định hướng của đạo hàm bậc hai.

TÀI LIỆU THAM KHẢO

- [1] R. V. Kashyap, “A survey of deep learning optimizers: First and second-order methods,” *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 37, no. 4, Art. no. 111, Aug. 2023.
- [2] M. Yousefi and Á. Martínez, “Deep neural networks training by stochastic quasi-Newton trust-region methods,” *Algorithms*, vol. 16, no. 10, Art. no. 490, 2023.
- [3] J. Martens, “Deep learning via Hessian-free optimization,” in *Proc. 27th Int. Conf. Mach. Learn. (ICML)*, Haifa, Israel, 2010, pp. 735–742.
- [4] Y. Ren, A. Bahamou, and D. Goldfarb, “Kronecker-factored quasi-Newton methods for deep learning,” *arXiv preprint arXiv:2102.06737*, Feb. 2022.
- [5] R. Shrestha, “Natural gradient methods: Perspectives, efficient-scalable approximations and analysis,” *arXiv preprint arXiv:2303.05473*, Mar. 2023.
- [6] R. Sun, “Optimization for deep learning: An overview,” *J. Oper. Res. Soc. China*, vol. 8, no. 2, pp. 249–294, Apr. 2020.
- [7] J. Martens, “*Second-order optimization for neural networks*,” Ph.D. dissertation, Dept. Comput. Sci., Univ. of Toronto, Toronto, ON, Canada, 2016.

Thông tin tác giả liên hệ: **Vũ Văn Đốc**

Điện thoại: 0912648561 - Email: vvdock@uneti.edu.vn

Khoa Công nghệ thông tin, Trường Đại học Kinh tế - Kỹ thuật Công nghiệp